LoRA Requisite Rank Experiment

Code Reference & Commentary

A line-by-line guide to the experiment codebase.
Covers design decisions, non-obvious invariants,
and the reasoning behind every major implementation choice.

Files covered

```
src/config.py        — constants, task and model registries
src/data_loader.py   — tokenisation and DataLoader construction
src/model.py         — LoRA / QLoRA model loading
src/train.py         — training loop, evaluation, checkpointing
run_experiment.py    — master runner, CLI, resume logic
analysis.py          — plots, heatmap, requisite rank table
```

Project Overview & Hypothesis

Core conjecture

Different NLP tasks require fundamentally different amounts of parameter change from a pretrained model. LoRA controls this precisely: rank r determines how many dimensions are available for adaptation. The experiment tests whether there is a task-specific threshold R* such that:

- Ranks below R* leave the model under-equipped — performance is noticeably worse than full fine-tuning.
- Ranks at or above R* match full fine-tuning performance.
- Crucially, R* is the same (or similar) across different model architectures, suggesting it is a property of the task's intrinsic dimensionality, not of the model's capacity.

Why train past convergence — the key design decision

NUM_EPOCHS = 10 is roughly 3x the training length at which RoBERTa typically converges on GLUE tasks. We deliberately train far beyond the performance peak. There are two reasons:

- We want to observe what happens after the optimum. If the rank is too low, the model will overfit within the low-rank subspace and performance will degrade visibly after the peak. If the rank is at or above R*, the low-rank constraint acts as a structural regulariser and performance remains stable — the model cannot overfit because it does not have enough degrees of freedom to memorise spurious training signal.
- Stopping early would give us only half the picture. The degradation curves after the peak are the actual experimental evidence, not a side-effect to be avoided.

In short: stable post-peak performance at rank r is the signature of R* having been reached. Visible degradation at rank r means r is too high and the model is starting to overfit into the LoRA subspace.

[!] The validation set is used purely for measurement — test metric is logged every 200 steps but never influences training decisions. There is no early stopping, no learning-rate gating on validation loss, and no best-checkpoint selection.

Experimental design at a glance

- 4 tasks: SST-2 (sentiment), CoLA (grammaticality), SNLI (inference), SQuAD 2.0 (extractive QA).
- 7 LoRA rank conditions: r = 1, 2, 4, 8, 16, 32, 64.
- 1 full fine-tuning baseline per task (upper bound).
- Phase 1: RoBERTa-base (125M encoder). Phase 2: RoBERTa-large, Llama-3.2-1B, Llama-3.2-3B.
- 32 training runs in Phase 1 (4 tasks x 8 conditions).
- Results saved per run: training_log.csv + model checkpoint.

Central configuration: constants, model registry, task registry

config.py

LoRA rank sweep constants

LORA_RANKS defines the seven conditions in the sweep. The values are powers of two, spanning a 64x range from minimal (r=1, 2 learned dimensions per weight matrix) to substantial (r=64, which begins to approach the dimension of the attention matrices themselves in RoBERTa-base). Equally-spaced doublings make the sweep visually interpretable on a log scale.

```
LORA_RANKS: List[int] = [1, 2, 4, 8, 16, 32, 64]

LORA_ALPHA_MULTIPLIER: float = 2.0
# alpha = rank * 2; effective LoRA output scale = alpha/rank = 2
# This is constant regardless of r, so rank comparisons are fair.

LORA_DROPOUT: float = 0.05

INCLUDE_FULL_PARAM_BASELINE: bool = True
# One fully unfrozen run per task as the performance ceiling.
```

*[!] LORA_ALPHA_MULTIPLIER is defined here for documentation purposes but the actual LoRA config in model.py uses lora_alpha = rank * 2 inline. The two must stay in sync if you change the multiplier.*

ModelConfig and the model registry

ModelConfig is a lightweight dataclass that stores everything the experiment needs to know about a base model. lora_target_modules differs between encoder and decoder architectures: RoBERTa exposes query/key/value/dense as separate named modules; Llama uses q_proj/v_proj. Including all four attention matrices for RoBERTa (not just Q and V) gives LoRA more representational surface and is consistent with the original LoRA paper's findings on encoder models.

```
ModelConfig(
    hf_name='roberta-base',
    params='125M',
    architecture='encoder',
    lora_target_modules=['query', 'key', 'value', 'dense'],
)
# Llama uses q_proj/v_proj only — the standard QLoRA setup.
# Decoder models skip K and dense to reduce adapter count.
```

TrainingConfig

TrainingConfig holds hyperparameters used by the DataLoader setup (batch_size) and is referenced by train.py through _training_cfg_for_task(). The actual learning rate, epoch count, and eval frequency are defined as module-level constants in train.py, not here — this keeps the training loop self-contained and easy to read.

[!] early_stopping_patience was deliberately removed from TrainingConfig. Leaving it in — even unused — would be misleading given that early stopping is explicitly forbidden by the experimental design.

Task definitions and SOTA baselines

Each TaskConfig entry defines everything the experiment needs for one benchmark: where to load the data, which columns to use, how to tokenize it, which metric to evaluate, and what the published performance ceiling is.

- SST-2: binary sentiment. Metric = accuracy. SOTA = 96.4. Short sentences (max_input_length=64).
- CoLA: grammatical acceptability. Metric = Matthews Correlation Coefficient (MCC), not accuracy. MCC handles class imbalance correctly — CoLA is ~70% acceptable, so a model predicting always-acceptable scores ~70% accuracy but near-zero MCC. SOTA = 63.6.
- SNLI: 3-way natural language inference (entailment / neutral / contradiction). Metric = accuracy. SOTA = 91.8. Sentence pairs, max 128 tokens. Note: rows with label=-1 (no annotator consensus) are filtered out in data_loader.py.
- SQuAD 2.0: extractive QA with unanswerable questions. Primary metric = F1, secondary = Exact Match. SOTA = 86.5 F1 / 83.7 EM. Contexts up to 512 tokens, max_input_length=384 with stride-128 sliding windows.

```
# sota_baseline is used in two places:
#   1. compute_requisite_rank() in analysis.py:
#      lowest rank with peak >= sota_baseline - 0.5
#   2. _print_run_result() in run_experiment.py:
#      prints the gap to baseline after each run.
```

Tokenisation and DataLoader construction for all four tasks

Architecture overview

The file exposes one public function: get_dataloaders(). Internally it dispatches to two different tokenisation strategies depending on task_type:

- classification — handles both single-sentence (SST-2, CoLA) and sentence-pair (SNLI) with one shared function.
- span_extraction — SQuAD 2.0 needs two separate tokenisers: one for training (computes token-level start/end positions) and one for evaluation (preserves character offsets for post-processing).

```
def get_dataloaders(task, tokenizer, training_cfg=TRAINING) -> dict:
    # Returns {"train": DataLoader,
    #          "eval":  DataLoader,
    #          "eval_dataset": Dataset | None}
    # eval_dataset is only non-None for SQuAD; it carries
    # offset_mapping and example_id needed by post-processing.
```

Sliding window stride for SQuAD 2.0

SQuAD contexts can exceed 500 words, far longer than RoBERTa's 512-token limit. The tokenizer splits each context into overlapping windows using return_overflowing_tokens=True.

```
# 128-token overlap between consecutive windows so answers that
# straddle a boundary appear fully inside at least one window.
_SQUAD_STRIDE = 128

tokenizer(
    question, context,
    max_length=384,
    truncation='only_second',   # only truncate context, never question
    stride=_SQUAD_STRIDE,
    return_overflowing_tokens=True,
    return_offsets_mapping=True,
)
```

overflow_to_sample_mapping maps each feature (window) back to its source example index, allowing the train tokeniser to look up the correct answer span.

CLS position as the 'no answer' label

For unanswerable questions and for answer spans that fall outside the current window, the model is trained to point both start and end pointers at position 0 (the CLS token). This is the SQuAD 2.0 standard convention.

```
# CLS (position 0) is the SQuAD 2.0 convention for 'no answer'.
cls_idx = tokenized['input_ids'][feat_idx].index(
    tokenizer.cls_token_id
)

if not answers['answer_start']:  # unanswerable
    start_positions.append(cls_idx)
    end_positions.append(cls_idx)
```

Context boundary detection and the overshoot pattern

Finding the token-level start and end of the answer requires knowing where the context tokens begin and

end within the feature (accounting for the question tokens and special tokens that precede them). sequence_ids() returns 0 for question tokens, 1 for context tokens, and None for specials.

```
ctx_start = next(i for i, s in enumerate(seq_ids) if s == 1)
ctx_end = len(seq_ids) - 1
while seq_ids[ctx_end] != 1:  # walk back past padding and EOS
    ctx_end -= 1

# Walk forward: loop overshoots by one, so we step back.
tok_start = ctx_start
while tok_start <= ctx_end and offsets[tok_start][0] <= start_char:
    tok_start += 1
start_positions.append(tok_start - 1)  # correct for overshoot

# Walk backward: same overshoot-by-one pattern.
tok_end = ctx_end
while tok_end >= ctx_start and offsets[tok_end][1] >= end_char:
    tok_end -= 1
end_positions.append(tok_end + 1)       # correct for overshoot
```

Eval tokenisation: preserving offset_mapping

The eval tokeniser does not compute start/end positions (those are not knowable at inference time). Instead it preserves offset_mapping so that during post-processing we can map predicted token indices back to character spans in the original context string. Non-context token offsets are zeroed to prevent false character span matches.

```
clean_offsets.append(
    [(s, e) if seq_ids[k] == 1 else (0, 0)
     for k, (s, e) in enumerate(offsets)]
)
# (0, 0) acts as a sentinel: _postprocess_squad in train.py
# skips any span endpoint with offset (0, 0).
```

Custom collate function for SQuAD eval

PyTorch's default collate function crashes when it encounters offset_mapping (a list of (int, int) tuples) or example_id (strings). These fields are only needed by post-processing code, not by the model, so we keep them as plain Python lists and stack only the tensor fields.

```
def _squad_eval_collate(batch):
    result = {}
    for key in batch[0]:
        values = [ex[key] for ex in batch]
        if isinstance(values[0], torch.Tensor):
            result[key] = torch.stack(values)  # normal tensor
        else:
            result[key] = values               # keep as list
    return result
```

[!] output_all_columns=True in ds.set_format() is also required — without it, HuggingFace's set_format silently drops any column not listed in the 'columns' argument, removing offset_mapping and example_id before the DataLoader ever sees them.

DataLoader settings

```
DataLoader(
    train_ds,
```

```
    batch_size=training_cfg.batch_size,
    shuffle=True,
    num_workers=4,
    pin_memory=True,   # async CPU->GPU transfer; free speedup on CUDA
)
```

LoRA and QLoRA model loading for all four supported architectures

Quantization map

Each model maps to a BitsAndBytesConfig (or None for no quantization). Phase 1 uses only roberta-base, so quantization is not active yet. The map is here for Phase 2.

```
_QUANT_MAP = {
    'roberta-base':             None,         # standard fp32
    'roberta-large':            None,         # standard fp32
    'meta-llama/Llama-3.2-1B': _BNBCONFIG_8BIT,
    'meta-llama/Llama-3.2-3B': _BNBCONFIG_4BIT,
}
```

4-bit quantization config (QLoRA)

The 4-bit config uses three specific choices that together constitute the QLoRA recipe from Dettmers et al. 2023:

```
_BNBCONFIG_4BIT = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    # ^ matmuls run in fp16 even though weights are stored in 4-bit
    bnb_4bit_quant_type='nf4',
    # ^ NormalFloat4: optimal for normally-distributed LLM weights
    bnb_4bit_use_double_quant=True,
    # ^ quantises the quantisation constants too (~0.4 bits/param saved)
)
```

prepare_model_for_kbit_training

This bitsandbytes utility must be called on any quantized model before LoRA adapters are attached. It does three things: casts LayerNorm layers to fp32 (for numerical stability during backprop), enables gradient checkpointing (reduces activation memory at the cost of recomputation), and disables caching in attention layers (incompatible with gradient checkpointing).

```
if _QUANT_MAP.get(model_name) is not None:
    # Casts LayerNorms to fp32 and enables gradient checkpointing;
    # required before LoRA adapters are attached to a quantized model.
    base = prepare_model_for_kbit_training(base)
```

LoRA configuration

```
LoraConfig(
    r=rank,
    lora_alpha=rank * 2,
    # ^ effective scale = alpha/r = 2; constant across all ranks.
    # Without this, a rank-64 model would have 64x larger output
    # than a rank-1 model, making comparisons unfair.
    lora_dropout=0.1,
    bias='none',
    # ^ training bias terms would break the low-rank structure
    target_modules=model_cfg.lora_target_modules,
    # ^ sourced from MODEL_REGISTRY so each architecture adapts
    # the correct projection matrices automatically
)
```

target_modules for RoBERTa is ['query', 'key', 'value', 'dense'] — all four weight matrices in each

self-attention block. For Llama it is ['q_proj', 'v_proj'] — the standard QLoRA subset. Using all four for RoBERTa gives LoRA more surface to work with and is consistent with encoder-model LoRA literature.

Full fine-tuning baseline

get_full_model() deliberately sets quantize=False even for Llama. bitsandbytes quantized weights are mathematically frozen — backprop cannot update them end-to-end. Loading without quantization gives a true full-parameter baseline, at the cost of higher memory usage.

```
def get_full_model(model_name, task_type, num_labels):
    # quantize=False: load Llama in fp16, not 4-bit/8-bit.
    # Quantized weights are frozen by design and cannot be
    # updated in a true full-parameter training run.
    model = _load_base_model(model_name, task_type,
                             num_labels, quantize=False)
    for param in model.parameters():
        param.requires_grad = True
    return model
```

Fixed-length training loop, SQuAD post-processing, checkpointing

Hyperparameter rationale

```
LR          = 2e-5   # standard GLUE/RoBERTa learning rate
WEIGHT_DECAY= 0.01
NUM_EPOCHS  = 10     # ~3x the typical 3-epoch GLUE convergence;
                     # ensures we run well past the performance peak
EVAL_EVERY  = 200    # dense enough for smooth degradation curves

BATCH_SIZE_CLS   = 32
BATCH_SIZE_SQUAD = 16  # SQuAD sequences are 384 tokens;
                       # half the batch to stay within memory

_N_BEST         = 20   # top-20 start/end pairs (standard SQuAD)
_MAX_ANSWER_LEN = 30   # spans > 30 tokens are almost certainly errors
_NULL_SCORE_DIFF= 0.0  # neutral: predict null only if it strictly wins
```

evaluate_classification — fresh metric per call

hf_evaluate metric objects accumulate predictions internally via add_batch(). If the same metric object were reused across eval calls, the second call would compute over both eval passes combined. Creating a fresh object each time ensures isolation.

```
def evaluate_classification(model, eval_loader, task, device):
    # Loaded fresh each call: HF metric objects accumulate state
    # internally, so reusing one across eval calls would corrupt
    # the running totals.
    metric = hf_evaluate.load(task.metric)
    # For CoLA: hf_evaluate.load('matthews_correlation')
    # For SST-2/SNLI: hf_evaluate.load('accuracy')
    ...
    return float(list(result.values())[0])
```

SQuAD post-processing: _postprocess_squad

SQuAD evaluation cannot be done token-by-token like classification. The model produces start and end logits over all tokens; we must convert these into text spans and compare against the reference answers at the character level. This happens in three passes:

- Group all features (windows) by their source example_id. One example may produce several overlapping features; they must be aggregated before making a single prediction.
- Compute the minimum null score across all features for the example. The null score is start_logit[0] + end_logit[0] (CLS position). We take the minimum because that is the window most confident the question is unanswerable.
- Compare the best non-null span score against the null score. If null_score - best_score > _NULL_SCORE_DIFF (0.0), predict empty string (unanswerable). Otherwise, extract the top-scoring character span from the context.

```
# Null score: both pointers at CLS (position 0)
null_score = float(start_logits[0] + end_logits[0])
# Take min across windows: most confident 'no answer' sets the bar.
min_null = min(min_null, null_score)

# Skip padding / question tokens (offset was zeroed in data_loader)
if offsets[s] == (0, 0) or offsets[e] == (0, 0):
    continue
```

```
if e < s or (e - s + 1) > _MAX_ANSWER_LEN:
    continue

# Predict unanswerable if null strictly wins
if min_null - best_score > _NULL_SCORE_DIFF:
    pred_text = ''
```

evaluate_squad returns (F1, Exact Match)

Both metrics are computed together from the squad_v2 evaluator. F1 is the primary metric (used for compute_requisite_rank). Exact Match is logged separately in the CSV and reported in the analysis summary table.

```
result = metric.compute(predictions=predictions,
                        references=references)
return float(result['f1']), float(result['exact_match'])
```

The training loop: key design choices

```
# Quantized models (Llama) placed by device_map='auto';
# calling .to() on them raises an error.
if not getattr(model, 'hf_device_map', None):
    model = model.to(device)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=int(0.06 * total_steps),
    num_training_steps=total_steps,
)  # schedule is a pure function of step count, not of validation loss

test_metric: float | str = ''
exact_match: float | str = ''
if global_step % EVAL_EVERY == 0:
    # Eval branch — purely for measurement, does not affect training
    ...
```

Final-step evaluation guarantee

If training ends on a step that is not a multiple of EVAL_EVERY, the last row in the CSV has no metric. We always evaluate at the very end so analysis.py always has a clean final data point.

```
# If training ended on a non-eval step the last CSV row has no
# metric; evaluate now so analysis always has a clean final point.
if global_step % EVAL_EVERY != 0:
    if task.task_type == 'classification':
        final_score = evaluate_classification(...)
        log_rows[-1]['test_metric'] = round(final_score, 4)
    else:
        final_f1, final_em = evaluate_squad(...)
        log_rows[-1]['test_metric'] = round(final_f1, 4)
        log_rows[-1]['exact_match'] = round(final_em, 4)
```

Checkpoint saving

After the log is written, the model is saved to results/{model_slug}/{task}/{rank}/checkpoint/. For LoRA runs, save_pretrained() writes only the adapter weights (adapter_model.safetensors + adapter_config.json) — typically a few MB. The full fine-tune run writes all 125M weights. The tokenizer is saved alongside so each checkpoint is self-contained and loadable independently.

```
ckpt_dir = os.path.join(out_dir, 'checkpoint')
model.save_pretrained(ckpt_dir)
tokenizer.save_pretrained(ckpt_dir)
```

Master runner: CLI, tokenizer setup, resume logic, ETA tracking

run_experiment.py

Condition ordering

Rank conditions are run cheapest-first: r=1, 2, 4, 8, 16, 32, 64, full. If a Colab session disconnects mid-sweep, the results already saved are the smallest and fastest runs — the most immediately useful partial data.

```
ALL_RANKS: list[str] = [str(r) for r in LORA_RANKS]
if INCLUDE_FULL_PARAM_BASELINE:
    ALL_RANKS.append('full')  # full fine-tune goes last — slowest run
```

CLI flags

```
python run_experiment.py                    # all 32 runs, roberta-base
python run_experiment.py --task sst2        # 8 conditions, SST-2 only
python run_experiment.py --rank 8           # rank-8 across all tasks
python run_experiment.py --task sst2 --rank full  # single run
python run_experiment.py --model roberta-large    # Phase 2 model
python run_experiment.py --device cpu       # force CPU (slow)
```

Tokenizer setup

The tokenizer is loaded once and shared across all runs for the chosen model. It is stateless after the initial setup, so one load avoids repeated HuggingFace hub calls. Decoder-only models (Llama) need two extra settings:

```
# Tokenizer is stateless after setup; one load serves all runs.
tokenizer = AutoTokenizer.from_pretrained(model_name)

if model_cfg.architecture == 'decoder':
    # Decoder models must pad on the left so causal attention
    # never attends to padding tokens that precede the sequence.
    tokenizer.padding_side = 'left'
    # Llama has no dedicated pad token; eos is the standard substitute.
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
        tokenizer.pad_token_id = tokenizer.eos_token_id
```

Resume support

run_summary.json is updated after every completed run and loaded at startup. Any run whose key appears with status='done' is skipped. This means a Colab session that disconnects after 10 of 32 runs can be restarted and will pick up from run 11 without re-running anything.

```
# Colab sessions disconnect mid-sweep; re-running the script
# picks up here and skips anything already marked 'done'.
if run_key in summary and summary[run_key].get('status') == 'done':
    print(f'[SKIP] {run_key} already complete')
    continue
```

[!] run_summary.json is model-scoped: stored at results/{model_slug}/run_summary.json. Runs for different models never interfere with each other's resume state.

Fresh model per run

```
# Fresh model every run: reusing would carry over learned weights
# from the previous rank condition and corrupt initialisation.
```

```
if rank_label == 'full':
    model = get_full_model(model_name, task.task_type, task.num_labels)
else:
    model = get_lora_model(int(rank_label), model_name,
                           task.task_type, task.num_labels)
```

ETA and error isolation

ETA is computed as the rolling average of all completed run times multiplied by the number of remaining runs. Each run is wrapped in try/except so a failed run prints its full traceback and records status='error' in the summary, then the loop continues to the next condition.

```
# ETA: average completed-run time x remaining runs
eta = (sum(completed_times) / len(completed_times)) * remaining

try:
    log_rows = train_one_run(...)
    status = 'done'
except Exception:
    traceback.print_exc()
    status = 'error'  # logged; loop continues to next run
```

Plots, cross-architecture heatmap, and requisite rank summary table

Three output types

- plots/{model_slug}/{task}_rank_sweep.png — test metric vs training step for all rank conditions on one task, one model. One plot per (model, task) pair.
- plots/{task}_cross_model_rstar.png — bar chart of R* across all four models for one task. Only produced when >= 2 models have results. This is the primary plot for testing the task-dimensionality hypothesis.
- plots/heatmap_requisite_rank.png — global heatmap with rows=tasks, columns=models, cells=R*. Gives the full picture at a glance.

REQUISITE_THRESHOLD = 0.5

```
# Half a metric point: small enough to be meaningful, large
# enough to absorb the evaluation variance typical of single-run
# GLUE / SQuAD experiments.
REQUISITE_THRESHOLD: float = 0.5

# R* = lowest rank r where:
#   peak_test_metric(r) >= sota_baseline - REQUISITE_THRESHOLD
```

Results path structure

load_results() globs results/{model_slug}/{task}/{rank}/training_log.csv. The model slug is the HuggingFace model ID with '/' replaced by '--' to be filesystem-safe (e.g. roberta-base, meta-llama--Llama-3.2-1B). The function returns a dict keyed by (model_slug, task, rank).

```
pattern = 'results/{model_glob}/{task_glob}/*/training_log.csv'
data[(m_slug, task, rank)] = df
# Only checkpoint rows are kept (test_metric != '')
# exact_match column is parsed if present (SQuAD only)
```

Rank sweep plot: plasma colormap sampling

```
def _rank_color(rank_label, n_lora_ranks):
    idx = lora_labels.index(rank_label)
    # Sample from 0.15-0.90 of plasma to avoid the near-white
    # and near-black extremes, which are hard to distinguish
    # on screen or when printed.
    t = 0.15 + 0.75 * (idx / max(n_lora_ranks - 1, 1))
    return RANK_CMAP(t)
```

Full fine-tuning is plotted as a dashed black line (distinct from all rank colours) and the SOTA baseline as a dotted crimson horizontal line. A shaded band covers the ±0.5-point threshold region for visual reference.

compute_requisite_rank

```
def compute_requisite_rank(task_data, sota_baseline):
    # Scan LORA_RANKS in ascending order (1, 2, 4, ...).
    # Return the FIRST rank whose peak test metric is within
    # 0.5 points of sota_baseline.
    for rank_label in [str(r) for r in LORA_RANKS]:
        peak = task_data[rank_label]['test_metric'].max()
        if peak >= sota_baseline - REQUISITE_THRESHOLD:
            return rank_label, peak  # lowest qualifying rank
    return None, None  # no rank reached the threshold
```

Heatmap: log2 scale and NaN colour

Rank values [1, 2, 4, 8, 16, 32, 64] are powers of two. A linear colormap would compress all the low-rank differences into near-zero while giving most of the colour range to the high end. log2 maps the ranks to equally-spaced integers [0, 1, 2, 3, 4, 5, 6], making every doubling the same visual step.

```
cmap = plt.get_cmap('RdYlGn_r').copy()  # green=low, red=high
# Grey for NaN (no data / no qualifying rank).
# White would be indistinguishable from the light end of the map.
cmap.set_bad(color='#d0d0d0')

# Log2 scale: vmin=log2(1)=0, vmax=log2(64)=6
# Each rank doubling = one equal visual step.
vmin, vmax = 0.0, float(np.log2(LORA_RANKS[-1]))

# Colorbar ticks are relabelled with the actual rank integers
# so the reader sees '1, 2, 4, ...' not '0, 1, 2, ...'
cbar.set_ticks([np.log2(r) for r in LORA_RANKS])
cbar.set_ticklabels([str(r) for r in LORA_RANKS])
```

Exact Match peak for SQuAD

```
# EM peak is independent of R*: we want the best EM the model
# achieved at any rank, not just at the requisite rank.
em_vals = [
    float(df['exact_match'].max())
    for df in task_data.values()
    if 'exact_match' in df.columns
    and not df['exact_match'].isna().all()
]
em_peak = max(em_vals) if em_vals else None
```

Summary table columns

```
Model          | Task               | Requisite R* |
Peak F1/metric | Exact Match        | SOTA baseline | Gap (F1)

# Exact Match column shows the peak EM for SQuAD rows.
# For SST-2 / CoLA / SNLI it shows '--' (not applicable).
# Gap (F1) = Peak F1/metric - SOTA baseline (negative = below SOTA).
```